DTU Compute
Department of Applied Mathematics and Computer Science

Thesis title
Thesis subtitle

Master Thesis

Thesis title
Thesis subtitle

Master Thesis
Date, year

By
Author

Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Approval

This thesis has been prepared over six months at the Section for Indoor Climate, Department of Civil Engineering, at the Technical University of Denmark, DTU, in partial fulfilment for the degree Master of Science in Engineering, MSc Eng.

It is assumed that the reader has a basic knowledge in the areas of statistics.

Author - s123456

.....

Signature

.....

Date

Abstract

There often appear cases where a particular marginal distribution in the generative process of a diffusion model is undesirable or better replaced with another distribution to suit a specific task. Scenarios such as fair generation or label shift are common examples of this need. Using Jeffrey's Rule, it is possible to transform this undesired distribution into a more suitable target distribution. In the context of distribution-guided diffusion, actually sampling from the posterior requires an accurate estimate of the clean data x_0 from a noisy sample x_t. Tweedie's formula is often employed for this purpose but its use of a single expected value, although good for simple classifier-guided diffusion, may not be best suited for a more complex case such as distribution-guidance. This thesis explores a Markov Chain Monte Carlo (MCMC) approach for sampling from the marginal distribution defined by Jeffreys Rule, investigating its performance within this distribution-guided diffusion setting. VERSION 2:

While diffusion models capture the data distributions they are trained on, the resulting learned marginals may require modification to satisfy constraints such as fairness or to account for label shift. Jeffrey's rule provides a principled mechanism for transforming an undesired marginal distribution into a desired target distribution.

Sampling from the resulting posterior during the diffusion process requires estimating the clean data x_0 from a noisy observation x_t. A common approach is to use Tweedie's formula, which provides an estimate of x_0 through the posterior mean. While effective in many settings, this mean-based approximation may be insufficient when the target distribution is complex or multi-modal, as can occur in distribution-guided diffusion models.

This thesis investigates the use of Markov Chain Monte Carlo (MCMC) methods to sample from the marginal distribution induced by Jeffrey's rule, evaluating their effectiveness within a distribution-guided diffusion framework.

Acknowledgements

Author, MSc Mathematical Modelling and Computation, DTU
Creator of this thesis template.

[Name], [Title], [affiliation]
[text]

[Name], [Title], [affiliation]
[text]

Contents

1 Introduction

Diffusion models, inspired by non-equilibrium statistical physics [1], are a popular paradigm of generative machine learning models that are prominently used for many different generation tasks such as image, video and audio synthesis as well as for scientific research tasks like 3D molecular generation and protein folding. [2] [3] [4] [5] [6] More recently they have also been adopted heavily in world simulations and physical modeling, which is used in areas such as robotics [7].

Training diffusion models means having them learn a distribution that matches the training data distribution as closely as possible. If the marginal distribution of the training data is somehow wrong, biased or doesn't reflect real world scenarios accurately then the generated samples of the trained model will reflect this in their output and give samples exhibiting this undesirable distribution. Even if the data distribution is fine there are still many situations where it is desirable to guide the generation process of the model toward some desired distribution, such as generating 50% male and 50% female images, or generating samples of particular colors.

Many different methods exist that allow for controlling the generation process. Incorporating the conditioning information into the training process [8] via classifier-guided diffusion or other similar conditional training approaches is one method but it requires training a task specific model along side large amounts of data, pairing the training data along the conditioning information. More flexible methods exist that allow for conditional sampling by directly working on the unconditional model. Twisted Diffusion Sampling [9] and Feynman-Kac steering [10] are recent examples of approaches that do not require task-specific training and allow for flexible control of the sampling process. Normally, TDS is done when the generation is conditioned on a fixed, single observation y, where y is defined through the likelyhood $p(y|x)$ [9], but combining it with a specific method for updating probability distributions, namely Jeffrey's rule, allows for steering toward a distribution instead. Feynman-Kac steering uses an arbitrary reward function to control the generation which naturally allows for steering toward a desired distribution.

Jeffrey's rule of conditioning is a generic way to update probability distributions in order to satisfy some given constraint. More specifically it says that the standard joint distribution

$$p(z_I, z_J) = p(z_I|z_J)p(z_J)$$

Can simply be rewritten as:

$$p^*(z_I, z_J) = p(z_I|z_J)p^*(z_J)$$

Where $p^*(z_J)$ is an adjusted marginal distribution of a subset of the features which had an undesirable marginal to begin with. This construction then leads to the formulation:

$$p^*(z_I, z_J) = \frac{p^*(z_J)}{p(z_J)}p(z_I, z_J)$$

Which says that the new joint distribution is an importance-weighted version of the original p. Combining Jeffrey's conditioning with the above mentioned methods for steering the sampling, it is possible to condition the sampling process on an arbitrary distribution. (CHECK? IS IT TRUE? FOR ANY ARBTRIARY DISTRIBUTION?)

This thesis explores using Jeffrey's rule CITATION needed(Jeffrey PDF from Jes? OR perhaps dig deeper into the refernces listed on there.) in combination with the algorithms for conditioning the sampling process of a diffusion model in order to steer the generation towards a desired target distribution. These methods will be evaluated and compared in terms of their sampling efficiency, distributional fidelity, and ability to maintain sample diversity while adhering to external constraints. (DOUBLE CHECK LAST SENTENCE, WHAT TO COMPARE?)

2 Background

This chapter gives the reader background into the theory that this thesis is based on. First, score-based diffusion models are explained via stochastic differential equations, which provides the core framework for the diffusion model implementation of the thesis. Secondly, the core workings of Jeffrey's rule of conditioning are layed down and how it can be used to modify a joint distribution to include a more desirable marginal. Finally, the essentials of some algorithms for doing distribution-guided generation are described.

2.1 Diffusion Models

2.2 Score-based Diffusion Models via SDEs

Diffusion models are probabilistic generative models that work by slowly corrupting the initial data into noise, and then learn to reverse this corrupted data to generate a clean sample.

Two successful classes of diffusion models, Denoising diffusion probabilistic models (DDPM) [2] and Score matching with Langevin dynamics (SMLD) [11] are referred to as score-based generative models since they either directly estimate the score (SMLD), which is the gradient of the log probability density with respect to the data, or they implicitly estimate it. A breakthrough in diffusion modelling occurred when these two methods were unified and shown to be different discretizations of Stochastic Differential Equations (SDEs) [12], a perspective that generalized these frameworks into a continuous-time diffusion process.

MAYBE TALK ABOUT DIFFUSION PROCESSES HERE? WHAT THE DEFINITION IS ETC (MARKOV CHAIN SATISFYING TWO PROPERTIES)

ALSO MAYBE HAVE SUBSECTIONS TALKING ABOUT DDPM AND SMLD IN MORE DETAIL?

Score-based generative modelling via SDEs uses an infinite number of noise scales to perturb the data such that it evolves according to an SDE. The diffusion process then becomes $\{\mathbf{x}(t)\}_{t=0}^{T}$, indexed by the continuous time variable $t \in [0, T]$ instead of being made up of a discrete, finite number of noise scales. This diffusion process is such that when t is equal to 0 it matches the training data distribution and when $t = T$ it matches some given prior distribution which is usually Gaussian noise with a fixed mean and variance. The reverse process is then modelled by the reverse-time SDE of the forward process, (MAYBE OMIT THIS PART? ->) which is itself a diffusion process [13]. The forward diffusion process can be formulated as the solution to an Itô SDE [14].

$$\mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{w} \tag{2.1}$$

where $f(x, t) : \mathbb{R}^d \to \mathbb{R}^d$ is a vector valued function which is named the *drift* coefficient of $\mathbf{x}(t)$, $g : \mathbb{R} \to \mathbb{R}$ is called the *diffusion* coefficient of $\mathbf{x}(t)$ and $\mathbf{w}$ is the Wiener process. The solution is a continuous stochastic process that starts equal to the data distribution at $t = 0$ and ends as noise when $t = T$. At time step t the process can be described by the probability density $p_t(\mathbf{x})$ and the transition from a previous state to the next can be described by the conditional density $p(\mathbf{x}(t)|\mathbf{x}(s))$ where $0 \leq s < t \leq T$.

The reverse-process can be modelled by the reverse-time SDE:

$$\mathrm{d}\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]\mathrm{d}t + g(t)\mathrm{d}\bar{\mathbf{w}} \tag{2.2}$$

The solution to this SDE is then a stochastic process such that when simulated yields samples at $t = 0$. The only obstacle is that the score, $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, is generally intractable and thus has to be estimated somehow. Score-matching [15] allows for predicting the score via a score-based model, $\mathbf{s}_\theta(\mathbf{x}, t)$, often a neural network which is trained via:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)} \left[\left\| \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p(\mathbf{x}(t)|\mathbf{x}(0)) \right\|_2^2 \right] \right\} \tag{2.3}$$

Where $\lambda : [0, T] \to \mathbb{R}^+$ is positive weighting function and we sample t uniformly from $[0, T]$. Note that for the score, the conditional density $p(x(t)|x(0))$ is used, instead of the marginal of $x(t)$, this is again because the marginal is generally hard to compute and a result from [16] shows that the scores are equal for the two distributions up to a constant. This method of score matching is called Denoising Score Matching since it uses the conditional of the noisy sample given the clean one to calculate the score. Once we have access to the score estimation, sampling can be done by using a general SDE solver and simulating the answer to the reverse SDE.

2.3 Jeffrey's rule of Conditioning

Jeffrey's rule of conditioning is a general method for changing a probability distribution to satisfy some given constraint. (AGAIN CHECK FOR CITATION, CAN I CITE THE JEFFREY'S PDF FROM JES?). Given the set $\mathcal{Z} = \mathcal{Z}_1 \times \mathcal{Z}_2 \times \ldots \times \mathcal{Z}_d$, where we assume that all the sets $\mathcal{Z}_1, \mathcal{Z}_2, \ldots, \mathcal{Z}_d$ are countable. Next, we have some probabilistic model which we describe as a probability distribution over $\mathcal{Z}$. We assume that the probability distribution has full support, meaning that there is a non-zero probability of each event occurring. This allows for writing the probability distribution as a strictly positive probability mass function $p : \mathcal{Z} \to (0, 1]$. TALK ABOUT MEASURE THEORETIC FORMULATION? OR MAYBE JUST MENTION IT LIKE IN THE JEFFREY PDF

2.4 Twisted Diffusion Sampling

Twisted Diffusion Sampling (TDS) is a sequential Monte Carlo (SMC) algorithm that allows for flexible conditional generation of samples [9], without needing a conditionally trained model specific to only a single conditional generation task. SMC algorithms are a general way of sampling from a sequence of distributions on variables $x^{0:T}$, terminating at some final target distribution. They work by generating an ensemble of K *particles* $\{x_k^t\}_{k=1}^K$ at timestep t of an iterative procedure which has T steps in total, often denoted as starting at timestep T and going down to 0. The main components of SMC are, weighting functions denoted by $w_T(x^T)$, $\{w_t(x^t, x^{t+1})\}_{t=0}^{T-1}$ and proposals, which are distributions and are denoted by $r^T(x^T)$, $\{r_t(x^t|x^{t+1})\}_{t=0}^{T-1}$, that are used at each step of the procedure. More definitively, the general recipe for an SMC algorithm is the following:
At initial timestep T; K particles are drawn $x_k^T \sim r_T(x^T)$ and the weights are set to $w_k^T := w_T(x_k^T)$, then repeat sequentially for $T - 1, \ldots, 0$:

resample: $\{x_k^{t+1:T}\}_{k=1}^K \sim$ Multinomial$(\{x_k^{t+1:T}\}_{k=1}^K; \{w_k^{t+1}\}_{k=1}^K)$
propose: $x_k^t \sim r_t(x^t|x^{t+1}), k = 1, \ldots, K$
weight: $w_k^t := w_t(x_k^t, x_k^{t+1}), k = 1, \ldots, K$

At each timestep t the weighting functions and proposals form an intermediate target distribution [9] denoted by ν_t and defined as:

$$\nu_t(x^{0:T}) := \frac{1}{\mathcal{L}_t} \left[r_T(x^T) \prod_{t'=0}^{T-1} r_{t'}(x^{t'} \mid x^{t'+1}) \right] \left[w_T(x^T) \prod_{t'=t}^{T-1} w_{t'}(x^{t'}, x^{t'+1}) \right]$$

With *normalization* constant $\mathcal{L}_t$. These intermediate distributions are the joint probability distributions of all the variables $x^{0:T}$ with the path from T to t already having been weighted. They are the product of all the proposals, which are distributions, and the weights from t to T. The constant $\mathcal{L}_t$ is so that they integrate to 1 and form a valid probability distribution. Marginalizing the variables $x^{1:T}$ out of the final distribution $\nu_0(x^{0:T})$, yields $\nu_0(x^0)$ which is the final target distribution of interest and is the distribution of x^0 given the entire weights of all the steps.

In the context of conditional generation with diffusion models, the goal is to generate samples from a desired conditional distribution, $p(x^0 \mid y)$. In this setting, by choosing suitable proposals and weighting functions, Sequential Monte Carlo (SMC) methods can be used to construct a sequence of distributions whose terminal distribution $\nu_0(x^0)$ matches this desired conditional target.

However, in this setting it is generally not feasible to calculate the terminal $\nu_0(x^0)$ analytically since it requires marginalizing out the variables $x^1 : T$ which requires evaluating an intractable, high-dimensional integral over the highly non-linear transition distributions parameterized by the neural network. The weighted particles form a discrete approximation to $\nu_t(x^t)$ at each timestep t:

$$\left(\sum_{k'=1}^{K} w_{k'}^t\right)^{-1} \sum_{k=1}^{K} w_k^t \delta_{x_k^t}(x^t) \tag{2.4}$$

Where δ is a Dirac measure. Under standard assumptions, this empirical approximation converges setwise to $\nu_t(x^t)$ as the number of particles $K \to \infty$ (Proposition 11.4 in [17], Appendix A.5 in [9]). Consequently, the SMC approximation can be made arbitrarily accurate, and therefore provides an arbitrarily accurate approximation of the desired conditional distribution.

For conditional generation the standard setup is that there is some observation y related to the clean data variable x^0 through the likelihood $p(y|x^0)$ which may be given by a classifier, a learned scoring function, or a problem-specific measurement model. This, together with the diffusion model $p_\theta(x^{0:T})$ defines a joint distribution $p_\theta(x^{0:T}, y) = p_\theta(x^{0:T})\, p(y \mid x^0)$ where y depends only on x^0, since the clean data x^0 is only needed for conditioning and not the noisy x^t. The goal of conditional generation is then to sample from $p_\theta(x^0 \mid y)$. By using the Markov chain properties of diffusion models and including the rest of the variables $x^{1:T}$ it is possible to rewrite the conditional as:

$$p_\theta(x^{0:T} \mid y) = \frac{p_\theta(x^{0:T}, y)}{p_\theta(y)} = \frac{1}{p_\theta(y)} \left[p(x^T) \prod_{t=0}^{T-1} p_\theta(x^t \mid x^{t+1}) \right] p_{y|x^0}(y \mid x^0) \tag{2.5}$$

This factorization naturally fits into the SMC framework and matches the definition of the intermediate distributions nicely. By setting the proposals as $r_T(x^T) = p(x^T)$ and $r_t(x^t \mid x^{t+1}) = p_\theta(x^t \mid x^{t+1})$ for $1 \leq t \leq T$, and the weighting functions as $w_t(x^t, x^{t+1}) = 1$ for $1 \leq t \leq T$ with $w_0(x^0, x^1) = p_{y|x^0}(y \mid x^0)$ the final target distribution becomes $\nu_0 = p_\theta(x^{0:T} \mid y)$ with normalizing constant $\mathcal{L}_t = p_\theta(y)$. This is equivalent to simulating the entire unconditional diffusion process and then simply weighting the trajectories according to how likely the observation y is given the final sample x_0. That is, this is simply importance sampling with proposal $p_\theta(x^{0:T})$ and final weight $w(x^{0:T}) = p(y \mid x^0)$, with the intermediate weights all set to 1. If the conditional $p_\theta(x^0 \mid y)$ is too unlike the marginal $p_\theta(x^0)$ then most of the samples from the proposals will have low likelihood and the weights will kill most of

the particles yielding only very few samples that are actually usable. Furthermore, a result from [18] says that the number of particles required for accurate estimation of $p_\theta(x^{0:T} \mid y)$ is exponential in $\mathrm{KL}[p_\theta(x^{0:T} \mid y) \parallel p_\theta(x^{0:T})]$. Meaning that if the conditional and marginal are too different, the KL divergence becomes large and the number of particles needed for effective sampling becomes very high.

A method called *twisting* is often used in order to reduce the amount of particles necessary for a good approximation of the final target distribution. [19] [20] [21]. Twisting uses a sequence of *twisting functions* that alter the naive proposals and weights such that the intermediate distributions become closer to the final target. There exists optimal twisting proposals, denoted as r_t^* that are defined by multiplying the naive proposals by $p_\theta(y \mid x^t)$, that is:

$$r_T^*(x^T) \propto p(x^T)\, p_\theta(y \mid x^T) \text{ and } r_t^*(x^t \mid x^{t+1}) \propto p_\theta(x^t \mid x^{t+1})\, p_\theta(y \mid x^t), \quad 0 \leq t < T. \tag{2.6}$$

By using these proposals an SMC sampler draws exact samples from $p_\theta(x^{0:T})$ since by Bayes rule we get:

$$r_T^*(x^T) = p_\theta(x^T \mid y) \quad \text{and} \quad r_t^*(x^t \mid x^{t+1}) = p_\theta(x^t \mid x^{t+1}, y), \quad 0 \leq t < T. \tag{2.7}$$

Sampling from $x^T \sim r_T^*(x^T)$ and $x^t \sim r^*(x^t \mid x^{t+1})$ for $t = T-1, \ldots, 0$ is then equal to sampling from $p_\theta(x^{0:T} \mid y)$ by the chain rule of probability and the factorization in (2.5). The law of total probability then gives that the final sample obtained is $x^0 \sim p_\theta(x^0 \mid y)$.

This poises one problem, generally the distributions $p_\theta(y \mid x^t)$ are not analytically tractable and so sampling from these twisted proposals is not generally easy to do. Instead TDS approximates the twisting functions with:

$$\tilde{p}_\theta(y \mid x^t) := p_{y|x^0}(y \mid \hat{x}_\theta(x^t)) \approx p_\theta(y \mid x^t),$$

and uses this approximation to define the twisted proposals as:

$$\tilde{p}_\theta(x^t \mid x^{t+1}, y) := \mathcal{N}\left(x^t; x^{t+1} + \sigma^2 s_\theta(x^{t+1}, y), \tilde{\sigma}^2\right), \tag{2.8}$$

$$\text{where} \quad s_\theta(x^{t+1}, y) := s_\theta(x^{t+1}) + \nabla_{x^{t+1}} \log \tilde{p}_\theta(y \mid x^{t+1}) \tag{2.9}$$

and the twisted weights as:

$$w_t(x^t, x^{t+1}) := \frac{p_\theta(x^t \mid x^{t+1}) \tilde{p}_\theta(y \mid x^t)}{\tilde{p}_\theta(y \mid x^{t+1}) \tilde{p}_\theta(x^t \mid x^{t+1}, y)}, \qquad t = 0, \ldots, T-1$$

3 Examples of figures, tables, equations and listings

In the following a bunch of examples of figures and tables have been made. There are advantages to using `tikZ` diagrams over excel diagrams. 1) the font and font size perfectly matches the document 2) the styling and colours are pre-defined to follow the design guide 3) the plots uses vector graphics which reduces the file size, reduces the compile time and looks sharp when zooming in. The possibilities are endless, look at the `pgfplots` gallery for inspiration: `http://pgfplots.sourceforge.net/gallery.html`. However there are still cases where I would recommend to insert a plot as a picture. For example if the plot contains a lot of data: a line graph with 1000 points takes a long time to compile.

Some tips if you want good looking diagrams or graphs which will be inserted as pictures (e.g. in a figure environment with \includegraphics): The main font is Arial. Use DTU colours as described in **??**. Use high quality pictures. Try to scale the diagram (picture) so the text size of the axis legends match the text size in this document.

Remember to change the label of your figures so there are no duplicate labels. A label should be placed below a caption or after a heading (fx after a \chapter).

3.1 Graphs and charts

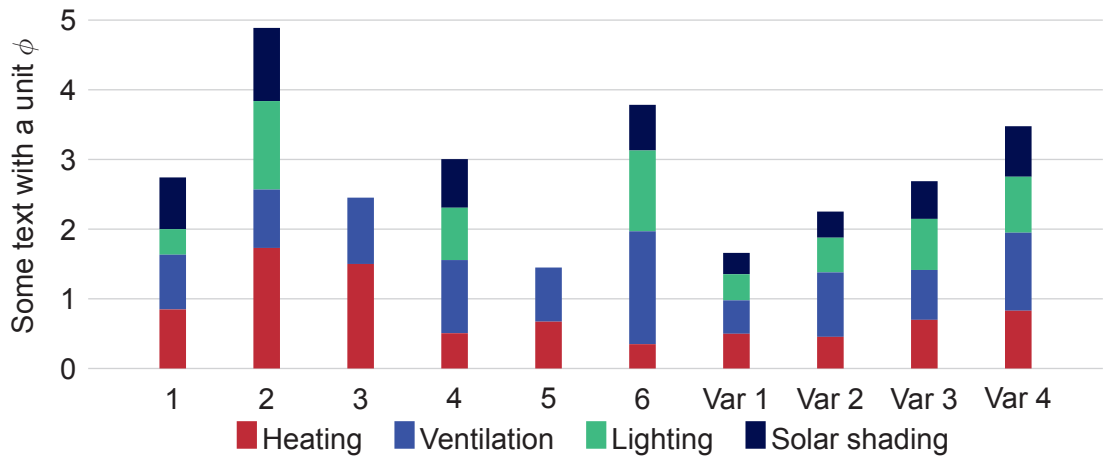

Figure 3.1: Stacked column chart

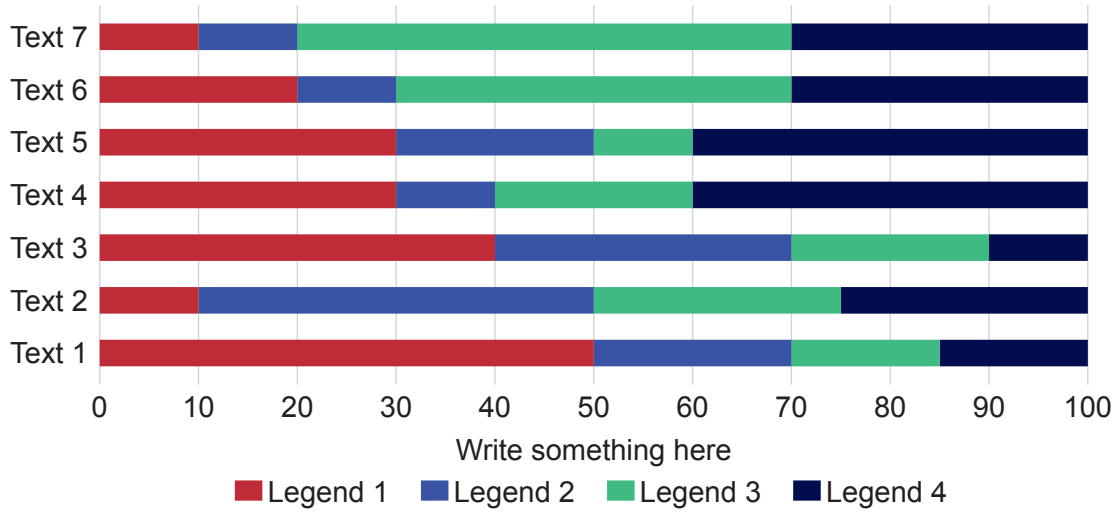

Figure 3.2: Stacked bar chart

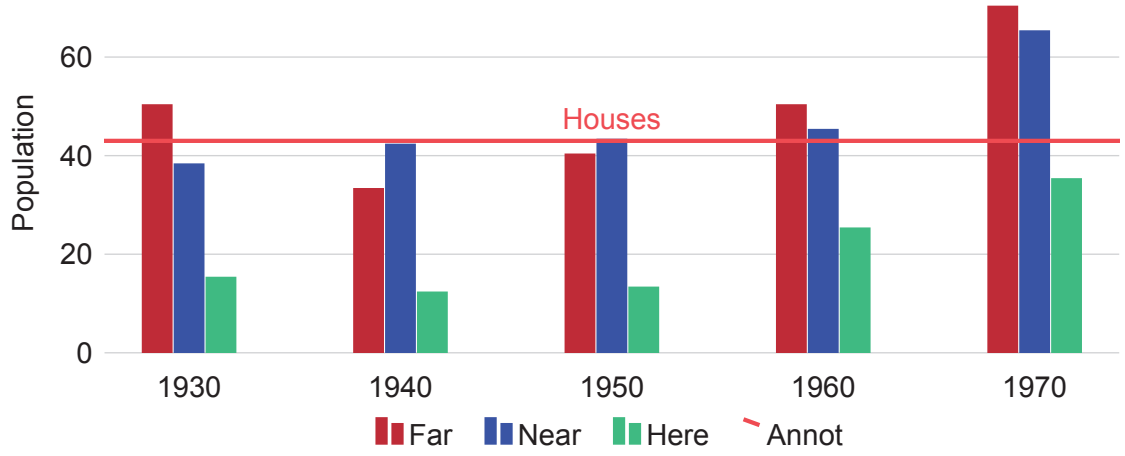

Figure 3.3: Grouped column chart

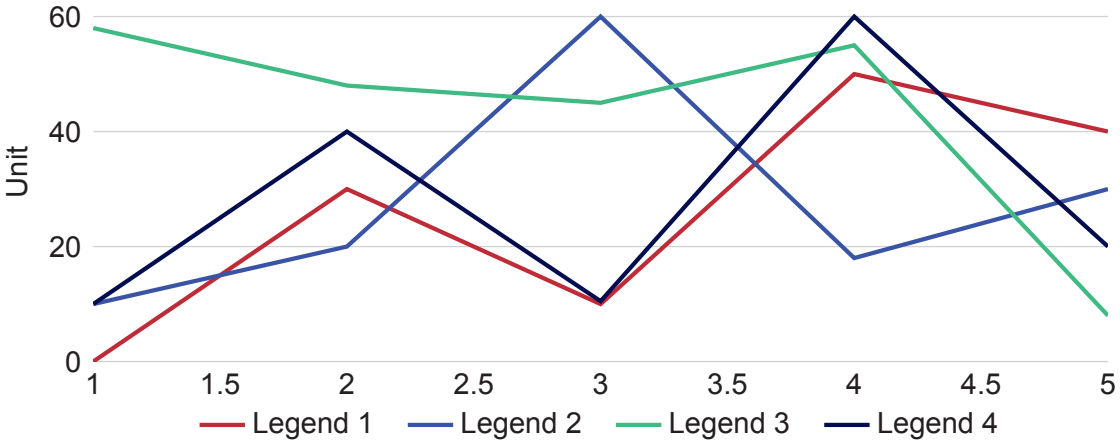

Figure 3.4: Line graph

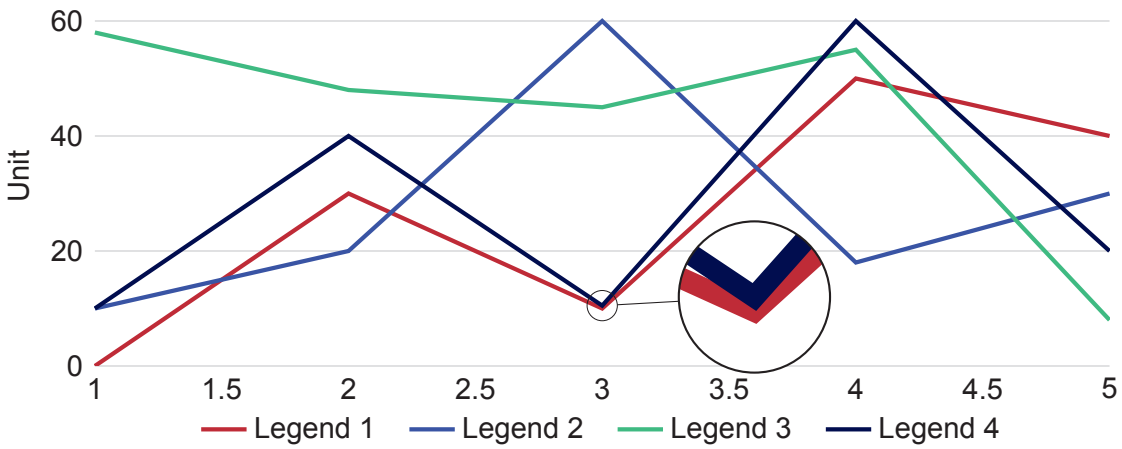

Figure 3.5: Line graph with magnifying glass

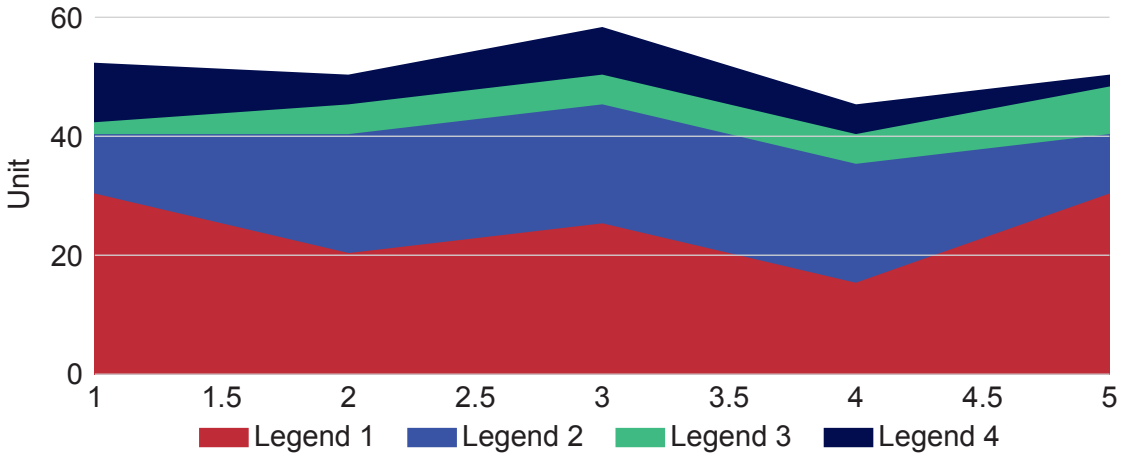

Figure 3.6: Area graph

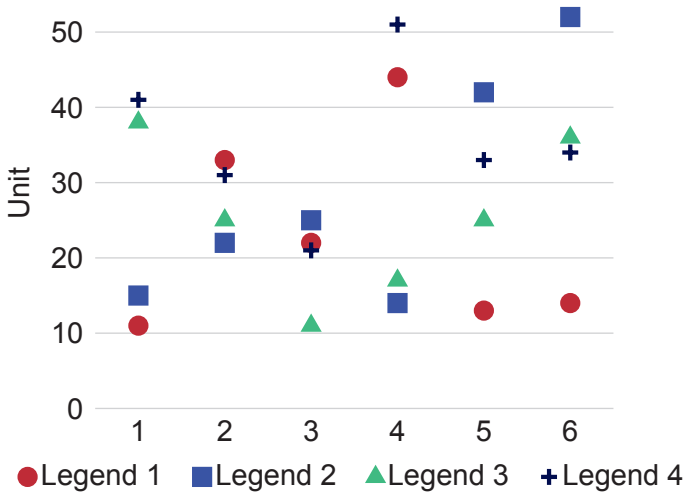

Figure 3.7: Scatter plot

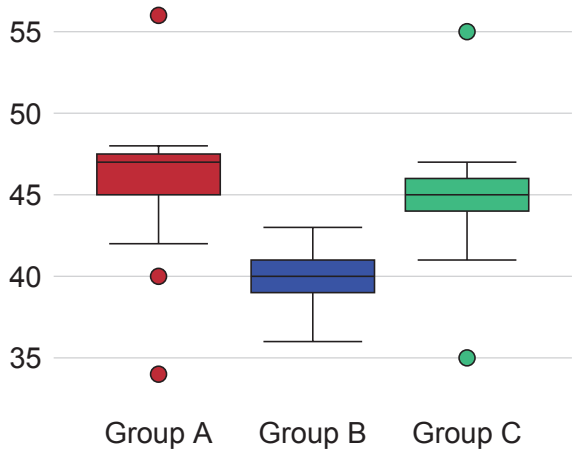

Figure 3.8: Boxplot

3.2 Tables and figures

Table 3.1: This is a `booktabs` table. Go to `http://www.tablesgenerator.com/` and use the booktabs table style

	Item	
Animal	Description	Price ($)
Gnat	per gram	13.65
	each	0.01
Gnu	stuffed	92.50
Emu	stuffed	33.33
Armadillo	frozen	8.99

`Booktabs` tables don't use any vertical lines. Only horizontal lines are used. Table 3.1 begins with a `\toprule`, ends with a `\bottomrule` with `\midrule` in between. The table has 3 columns formatted as `@{}llS@{}`. `@{}` is cropping the horizontal lines of the table to fit the content (removes column spacing at the left and right edges). `l` aligns the column to the left and `S` aligns the column according to the decimal point (`siunitx` package). You can of course also use `r` to align right or `c` to center the contents of the column.

Table 3.2: Wrongly formatted table

	Voltage V	Current A	Power W
Transformer input	234.4	0.50	117.4
Transformer output	25.86	2.72	70.3
Efficiency			60%

Table 3.3: Correctly formatted table

	Voltage V	Current A	Power W
Transformer input	234.4	0.50	117.4
Transformer output	25.86	2.72	70.3
Efficiency			60 %

Table 3.2 and table 3.3 have the same comtents but there are some subtle differences in formatting which makes table 3.3 the superior table of the two. The most obvious change is removing the midrule between the transformer input and output rows. The efficency row is the odd man out and a midrule has been used to emphasise the difference between the transformer rows and the efficiency row. The delimiters in the voltage, current and power columns are aligned. The horizontal lines (rules) fits to the content and instead of protruding. The spacing between 60 and the percentage sign is correctly adjusted.

Figure 3.9: Just a normal figure

(a) A subfigure

(b) A subfigure

Figure 3.10: A figure with two subfigures

(a) A subfigure

(b) A subfigure

(c) A subfigure

(d) A subfigure

Figure 3.11: A figure with four subfigures

Referring to the figure as a whole fig. 3.11 or to an individual sub figure fig. 3.11a is done the normal way with `\cref{}` commands.

3.3 Equations

In-line math is easy. Anything surrounded by dollar signs becomes a math field. Here is an example: $f(x) = 2x - 1$. Also anything inside the "\begin{equation}" and "\end{equation}" environment is also a math field. Examples are shown below.

All equations use the default latex font. Some might say it looks weird with a serif font for equations and a sans-serif font for the body text. However, it is very unpractical to change the math font in latex which is the exactly the reason why this has not been done. One benefit of the serif style math font is the clear distinction between symbols (variables) and units.

On the subject of units, those are all taken care of by the `\siunitx` package. Whenever there is a number followed by a unit one should write \SI{number}{unit}. Note this command is case sensitive. If a unit should follow a variable use the command \si{unit} (also case sensitive).

The ideal gas law is shown in eq. (3.1).

$$p \cdot V = n \cdot R \cdot T \tag{3.1}$$

$$\frac{\partial}{\partial t} \int_0^\delta U dy = -\delta \frac{1}{\rho} \frac{\partial P}{\partial x} - U_f(t)^2 \tag{3.2}$$

$$d_{step} = \sqrt{\frac{\delta}{\frac{dw}{dp_v}} \cdot t} = \sqrt{\frac{1.0 \times 10^{-11}\,\mathrm{kg/(m\,s\,Pa)}}{\frac{5.4\,\mathrm{kg/m^3}}{233.82\,\mathrm{Pa}}} \cdot 7200\,\mathrm{s}} = 0.001\,766\,\mathrm{m} = 1.766\,\mathrm{mm} \tag{3.3}$$

$$x = \mathtt{x}, \mathbf{x}, \boldsymbol{x}, x_{1_{2_{3_4}}}^{1^{2^{3^4}}} \cdot hello * \text{hello world} \cdot \text{equation without number}$$

Notice how the `aligned` environment can be used to align the equilibrium arrows in eq. (3.4). Only one equation number is generated using this method. Alternatively if you want an equation number for each line see eqs. (3.5) to (3.6).

$$\begin{aligned} CH_3COOH + OH^- &\rightleftharpoons CH_3COO^- + H_2O \\ H_2O &\rightleftharpoons H^+_{(aq)} + OH^-_{(aq)} \end{aligned} \tag{3.4}$$

$$f(x) = 1 + x - 3x^2 \tag{3.5}$$

$$g(x) + y = 3x - \frac{1}{2}x^3 \tag{3.6}$$

3.4 Listings (code)

Listing 3.1 is a nicely formatted block of code. A listing will automatically continue on the next page if it encounters a page break. Many different programming languages can be highlighted. Check the `listings` package documentation for a list of supported programming languages.

```
%% Monte Carlo simulation, estimation of pi
m=1E7;

x=rand(m,1);
y=rand(m,1);

g = x.^2+y.^2-1;

%dots outside
Pf = sum((g)<=0)/m

pi = 4*Pf
```

Listing 3.1: Monte Carlo simulation to estimate the value of π

Bibliography

[1] Jascha Sohl-Dickstein et al. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. arXiv: 1503.03585 [cs.LG]. URL: https://arxiv.org/abs/1503.03585.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: https://arxiv.org/abs/2006.11239.

[3] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: https://arxiv.org/abs/2112.10752.

[4] Jonathan Ho et al. *Video Diffusion Models*. 2022. arXiv: 2204.03458 [cs.CV]. URL: https://arxiv.org/abs/2204.03458.

[5] Zhifeng Kong et al. *DiffWave: A Versatile Diffusion Model for Audio Synthesis*. 2021. arXiv: 2009.09761 [eess.AS]. URL: https://arxiv.org/abs/2009.09761.

[6] Minkai Xu et al. *GeoDiff: a Geometric Diffusion Model for Molecular Conformation Generation*. 2022. arXiv: 2203.02923 [cs.LG]. URL: https://arxiv.org/abs/2203.02923.

[7] Ze Yang et al. *UniSim: A Neural Closed-Loop Sensor Simulator*. 2023. arXiv: 2308.01898 [cs.CV]. URL: https://arxiv.org/abs/2308.01898.

[8] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv: 2105.05233 [cs.LG]. URL: https://arxiv.org/abs/2105.05233.

[9] Luhuan Wu et al. *Practical and Asymptotically Exact Conditional Sampling in Diffusion Models*. 2024. arXiv: 2306.17775 [stat.ML]. URL: https://arxiv.org/abs/2306.17775.

[10] Raghav Singhal et al. "A General Framework for Inference-time Scaling and Steering of Diffusion Models". In: *Forty-second International Conference on Machine Learning*. 2025. URL: https://openreview.net/forum?id=Jp988ELppQ.

[11] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG]. URL: https://arxiv.org/abs/1907.05600.

[12] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456 [cs.LG]. URL: https://arxiv.org/abs/2011.13456.

[13] Brian D.O. Anderson. "Reverse-time diffusion equation models". In: *Stochastic Processes and their Applications* 12.3 (1982), pp. 313–326. ISSN: 0304-4149. DOI: https://doi.org/10.1016/0304-4149(82)90051-5. URL: https://www.sciencedirect.com/science/article/pii/0304414982900515.

[14] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. 6th. Universitext. Berlin Heidelberg: Springer-Verlag, 2010. ISBN: 978-3-642-14394-6. DOI: 10.1007/978-3-642-14394-6.

[15] Aapo Hyvärinen. "Estimation of Non-Normalized Statistical Models by Score Matching". In: *J. Mach. Learn. Res.* 6 (Dec. 2005), pp. 695–709. ISSN: 1532-4435.

[16] Pascal Vincent. "A Connection Between Score Matching and Denoising Autoencoders". In: *Neural Computation* 23.7 (2011), pp. 1661–1674. DOI: 10.1162/NECO_a_00142.

[17] Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Cham: Springer, 2020. ISBN: 978-3-030-47845-2. DOI: 10.1007/978-3-030-47845-2.

[18] Sourav Chatterjee and Persi Diaconis. *The sample size required in importance sampling*. 2017. arXiv: 1511.01437 [math.PR]. URL: https://arxiv.org/abs/1511.01437.

[19] Pieralberto Guarniero, Adam M. Johansen, and Anthony Lee. *The iterated auxiliary particle filter*. 2016. arXiv: 1511.06286 [stat.CO]. URL: https://arxiv.org/abs/1511.06286.

[20] Jeremy Heng et al. "Controlled sequential Monte Carlo". In: *The Annals of Statistics* 48.5 (Oct. 2020). ISSN: 0090-5364. DOI: 10.1214/19-aos1914. URL: http://dx.doi.org/10.1214/19-AOS1914.

[21] Christian A. Naesseth, Fredrik Lindsten, and Thomas B. Schön. *Elements of Sequential Monte Carlo*. 2024. arXiv: 1903.04797 [stat.ML]. URL: https://arxiv.org/abs/1903.04797.

A Title

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk